

```
# Cài đặt các thư viện cần thiết, bỏ qua lỗi nếu có
!pip install -q datasets requests torch peft bitsandbytes transformers trl accelerate sentence
```

Cài đặt các thư viện quan trọng để huấn luyện và chạy mô hình.

-q: Chạy lệnh cài đặt mà không hiển thị quá nhiều thông tin.

ignore the error message!: Bỏ qua lỗi (nếu có), do một số thư viện có thể đã được cài đặt.

```
# Import các thư viện cần thiết
import os
import re
import math
from tqdm import tqdm
from google.colab import userdata
from huggingface_hub import login
import torch
import transformers
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig, TrainingArguments
from peft import LoraConfig, PeftModel
from datasets import load_dataset, Dataset, DatasetDict
from datetime import datetime
import matplotlib.pyplot as plt
```

os, re, math: Xử lý chuỗi, toán học, hệ thống tệp.

tqdm: Hiển thị tiến trình khi chạy vòng lặp.

google.colab.userdata: Lấy thông tin người dùng trong Google Colab.

huggingface_hub.login: Đăng nhập vào Hugging Face.

torch, transformers: Xử lý mô hình học sâu và NLP.

datasets: Xử lý dữ liệu huấn luyện.

matplotlib.pyplot: Vẽ biểu đồ.

```
# Các mô hình Tokenizer có sẵn
LLAMA_3_1 = "meta-llama/Meta-Llama-3.1-8B"
QWEN_2_5 = "Qwen/Qwen2.5-7B"
GEMMA_2 = "google/gemma-2-9b"
PHI_3 = "microsoft/Phi-3-medium-4k-instruct"
```

Định nghĩa các mô hình phổ biến để lựa chọn.

```
# Định nghĩa các hằng số (constants)
BASE_MODEL = LLAMA_3_1 # Mô hình cơ sở được sử dụng
HF_USER = "ed-donner" # Người dùng trên Hugging Face
DATASET_NAME = f"{HF_USER}/pricer-data" # Tên bộ dữ liệu
MAX_SEQUENCE_LENGTH = 182 # Độ dài tối đa của một chuỗi đầu vào
QUANT_4_BIT = True # Sử dụng nén 4-bit hay không
```

Cấu hình mô hình và dữ liệu.

```
# Mã màu để in kết quả đẹp hơn
GREEN = "\033[92m"
YELLOW = "\033[93m"
RED = "\033[91m"
RESET = "\033[0m"
COLOR_MAP = {"red": RED, "orange": YELLOW, "green": GREEN}
```

Định nghĩa các màu sắc để hiển thị kết quả trực quan hơn.

```
%matplotlib inline
```

Hiển thị biểu đồ ngay trong notebook.

```
# Đăng nhập vào Hugging Face
hf_token = userdata.get('HF_TOKEN')
login(hf_token, add_to_git_credential=True)
```

Lấy mã truy cập (HF_TOKEN) từ dữ liệu người dùng.

Đăng nhập vào Hugging Face để tải mô hình.

```
# Kiểm tra Tokenizer của mô hình
def investigate_tokenizer(model_name):
    print("Investigating tokenizer for", model_name)
    tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
    for number in [0, 1, 10, 100, 999, 1000]:
        tokens = tokenizer.encode(str(number), add_special_tokens=False)
        print(f"The tokens for {number}: {tokens}")

# Kiểm tra Tokenizer của PHI_3
investigate_tokenizer(PHI_3)
```

Tải Tokenizer của mô hình được chọn.

Mã hóa một số số (0, 1, 10, 100, 999, 1000) thành token để kiểm tra cách mô hình xử lý chúng

```
# Tải bộ dữ liệu từ Hugging Face
dataset = load_dataset(DATASET_NAME)
train = dataset['train']
test = dataset['test']

# Xem một mẫu dữ liệu trong tập test
test[0]
```

Tải bộ dữ liệu từ Hugging Face.

Phân chia thành tập huấn luyện (train) và kiểm tra (test).

Hiển thị một mẫu từ tập kiểm tra.

```
# Chọn loại nén mô hình
if QUANT_4_BIT:
    quant_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_use_double_quant=True,
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_quant_type="nf4"
    )
else:
    quant_config = BitsAndBytesConfig(
        load_in_8bit=True,
        bnb_8bit_compute_dtype=torch.bfloat16
    )
```

Chọn nén 4-bit hoặc 8-bit để tối ưu bộ nhớ khi tải mô hình

```
# Tải Tokenizer và mô hình
tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"

base_model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL,
    quantization_config=quant_config,
    device_map="auto",
)
base_model.generation_config.pad_token_id = tokenizer.pad_token_id

print(f"Memory footprint: {base_model.get_memory_footprint() / 1e9:.1f} GB")
```

Tải Tokenizer và thiết lập ký tự padding (pad_token).

Tải mô hình đã được nén theo cấu hình (quant_config).

Hiển thị dung lượng bộ nhớ mô hình chiếm dụng.

```
# Hàm trích xuất giá tiền từ văn bản
def extract_price(s):
    if "Price is $" in s:
        contents = s.split("Price is $")[1]
        contents = contents.replace(',', '').replace('$', '')
        match = re.search(r"[-+]?[0-9]*\.[0-9]+\d+", contents)
        return float(match.group()) if match else 0
    return 0

extract_price("Price is $999 blah blah so cheap")
```

Xử lý văn bản để trích xuất giá tiền từ một câu.

```
# Hàm dự đoán giá tiền từ mô hình
def model_predict(prompt):
    set_seed(42)
    inputs = tokenizer.encode(prompt, return_tensors="pt").to("cuda")
    attention_mask = torch.ones(inputs.shape, device="cuda")
    outputs = base_model.generate(inputs, max_new_tokens=4, attention_mask=attention_mask, num_return_sequences=1)
    response = tokenizer.decode(outputs[0])
    return extract_price(response)

model_predict(test[0]['text'])
```

Dự đoán giá tiền từ văn bản bằng mô hình.

```
# Lớp kiểm thử mô hình
class Tester:
    ...
    Tester.test(model_predict, test)
```

Tester là lớp dùng để đánh giá độ chính xác của mô hình.

Kiểm tra độ sai lệch giữa giá dự đoán và giá thực tế.

Vẽ biểu đồ so sánh giữa giá thực tế và giá mô hình dự đoán.

Tóm tắt

Cài đặt và import thư viện.

Xác định mô hình và dữ liệu.

Đăng nhập Hugging Face, tải mô hình và Tokenizer.

Kiểm tra Tokenizer.

Tải dữ liệu, chọn phương pháp nén mô hình.

Xây dựng mô hình dự đoán giá từ văn bản.

Kiểm thử mô hình bằng lớp Tester.